



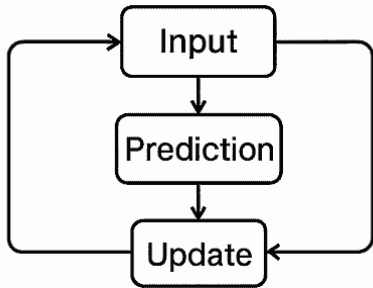
APPLIED DEEP LEARNING

06 – REINFORCEMENT LEARNING (RL)

PROF. DR. LARS ACKERMANN-IGL

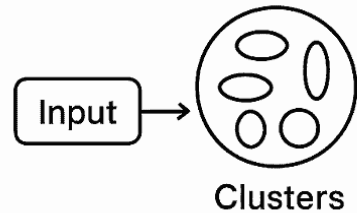
RECAP: BASIC TERMS OF MACHINE LEARNING (ML)

SUPERVISED LEARNING



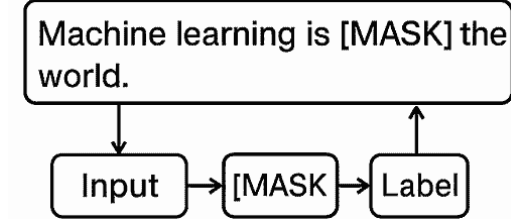
Supervised learning methods use **labeled** data (e.g. images of cats/dogs) as input and train a model in order to **minimize** the **classification error**. Labeled data is typically **split** into a **train** and **test** (and evaluation) set.

UNSUPERVISED LEARNING



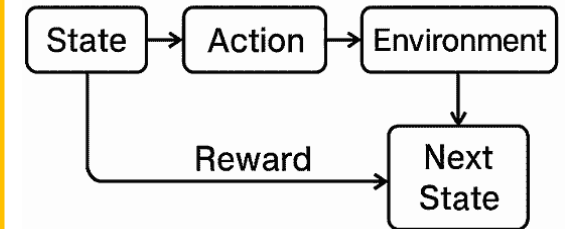
Unsupervised learning methods use **unlabeled** data as input and aim to **find groups / clusters** of similar data elements (with respect to a defined **distance metric**). Unsupervised methods can only detect **similarity** (no classes).

SELF-SUPERVISED LEARNING



Self-supervised learning methods use **unlabeled data** but create **pseudo-labels** from the data itself (e.g., predicting masked parts). The goal is to learn useful **representations** that can be used for downstream tasks.

REINFORCEMENT LEARNING



Reinforcement learning methods are e.g. often used within robotics / autonomous driving. Here (AI) models take **states** of the environment and possible **actions** as input and calculate an action as output, which has the **highest reward**.

In this course, we will focus primarily on supervised learning and cover unsupervised learning and reinforcement learning in less detail.

MOTIVATION: WHEN SUPERVISED LEARNING IS NOT FEASIBLE

Applications

- (Atari) Games / AlphaGo / ...
- Autonomous driving, robots, drones, ...
- Supply chain and logistics optimization, resource management, ...
- LLMs!
- ...

Why supervised learning is not feasible: Target outputs unknown or impractical to obtain

- Outcome-centric judgement: We only know the achievement but cannot judge the individual actions, e.g., in a football game ... pass or run? Depends on the the final result!
- Delayed judgement: Chess positions → What is the correct move? Rewards are delayed here!
- Long-term consequences: One action changes also the **future** state of an environment, e.g., in robot movements.



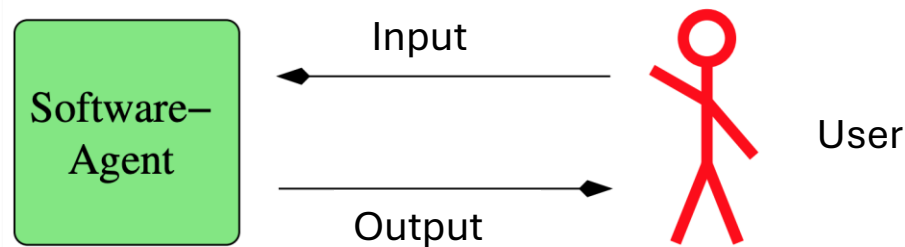
GENERALIZING FROM SPECIFIC ACTING ENTITIES TO AGENTS

Sensing and acting

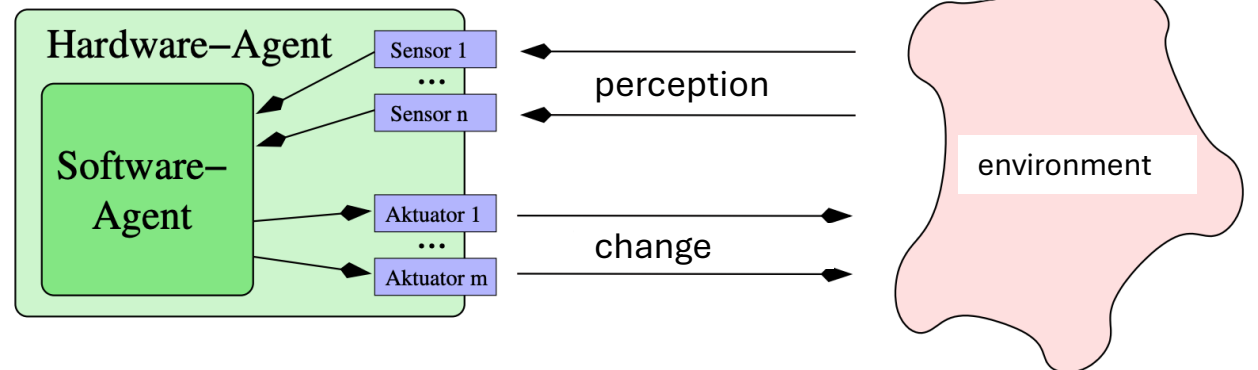
Agents ...

- Capture their environment through sensors (e.g., cameras, microphones, temperature sensors, GPS receivers, touch sensors).
- Perform actions in/on the environment through actuators (e.g., electric motors, robot arms, speakers, displays, grippers)

Software Agent



Hardware Agent

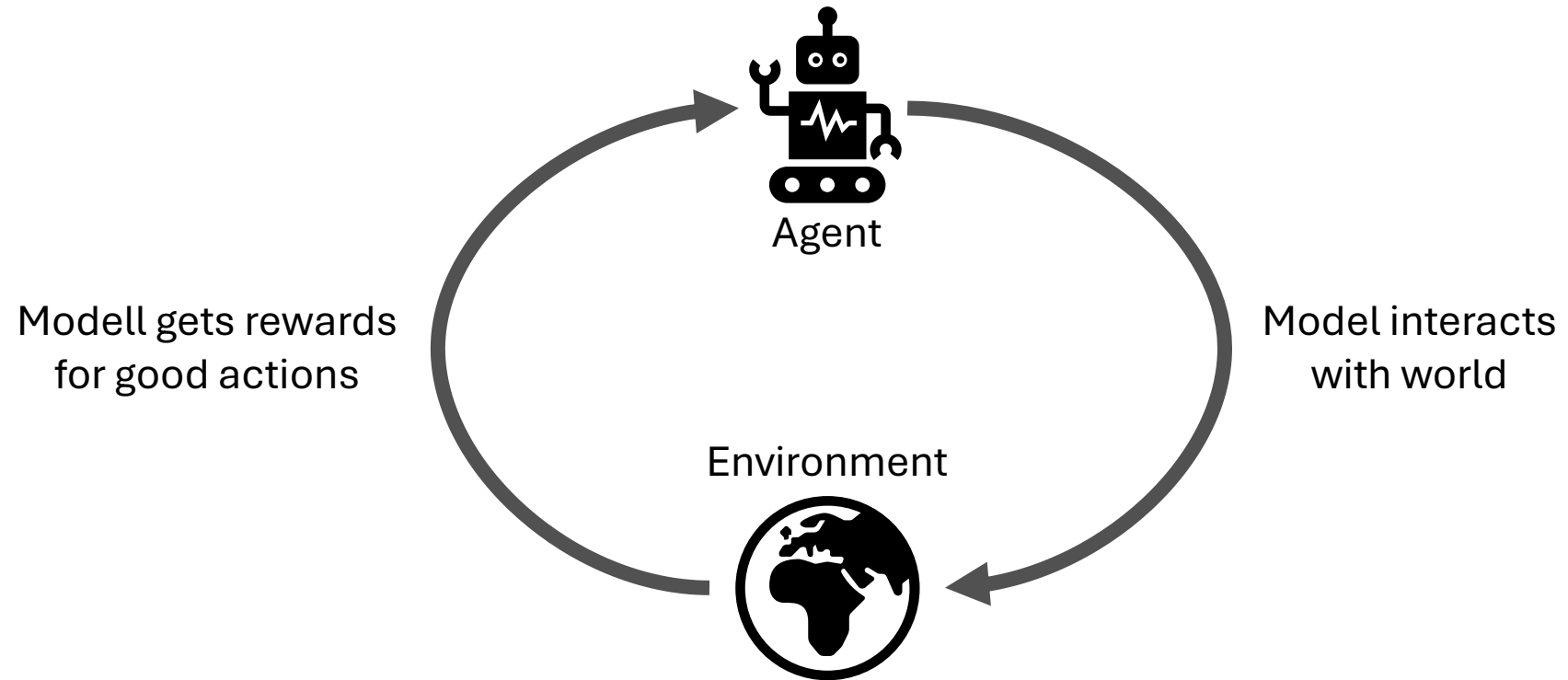




REINFORCEMENT LEARNING

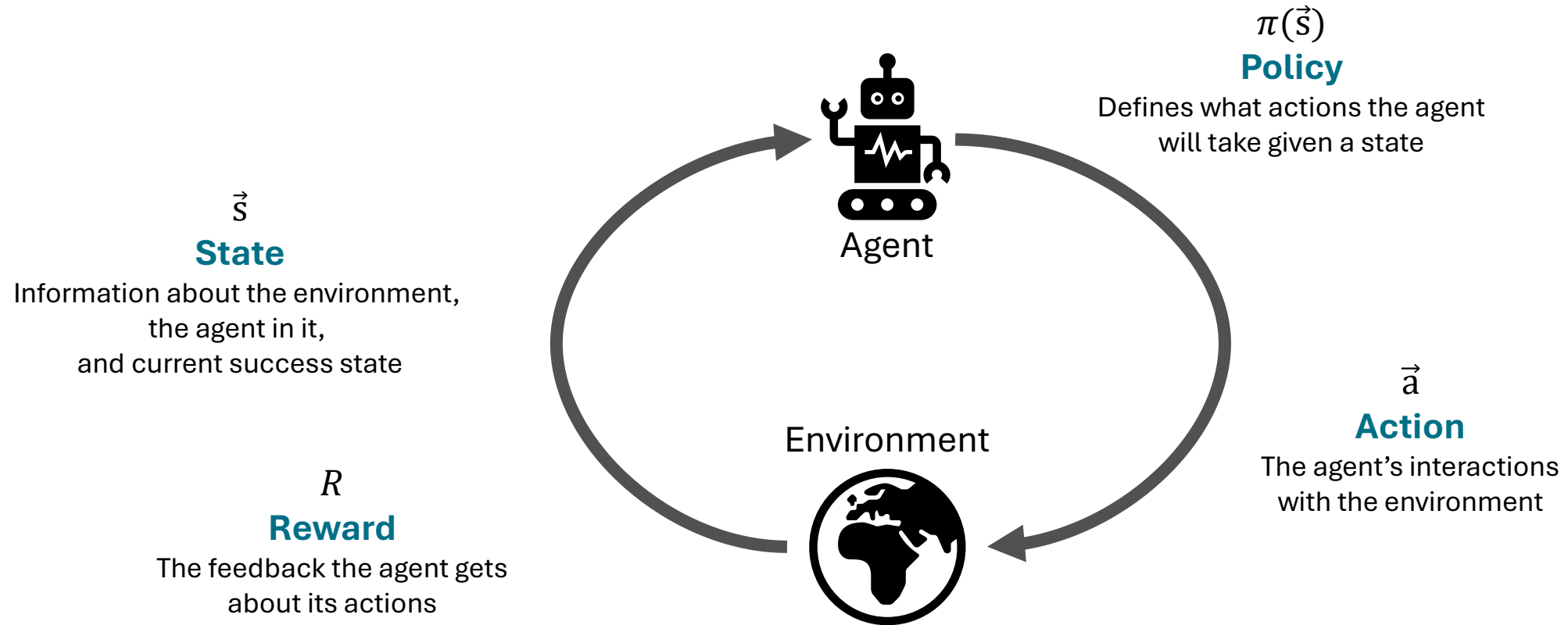
- Components: Agents, Policies, Environment, Actions, Rewards
- Detailing agents and environments

CORE PRINCIPLE – 1

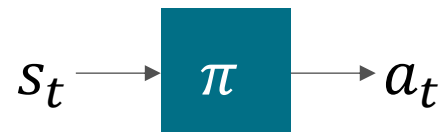


Reinforcement learning is based on trial and error!

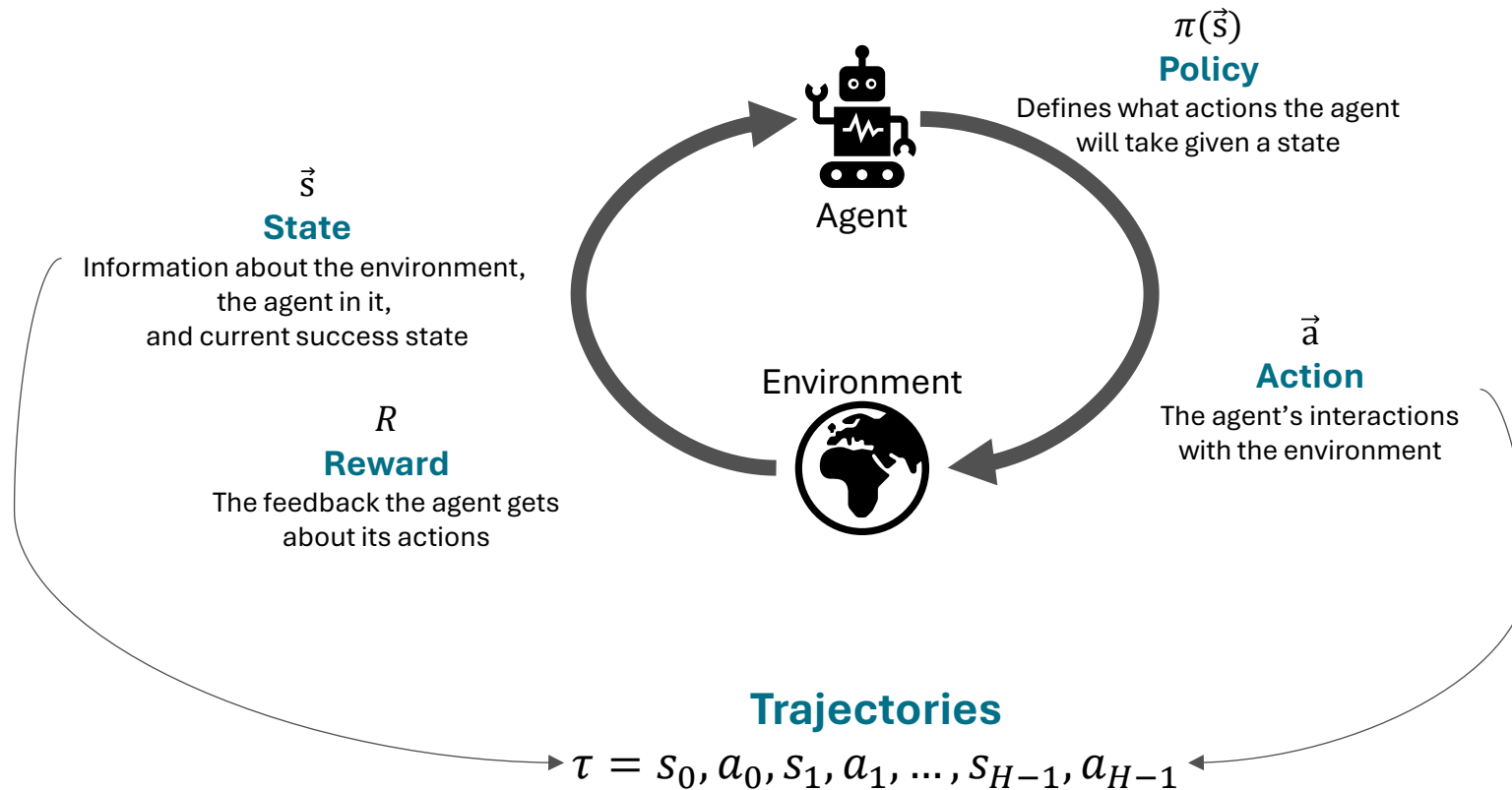
CORE PRINCIPLE – 2



Use of the policy



CORE PRINCIPLE – 3

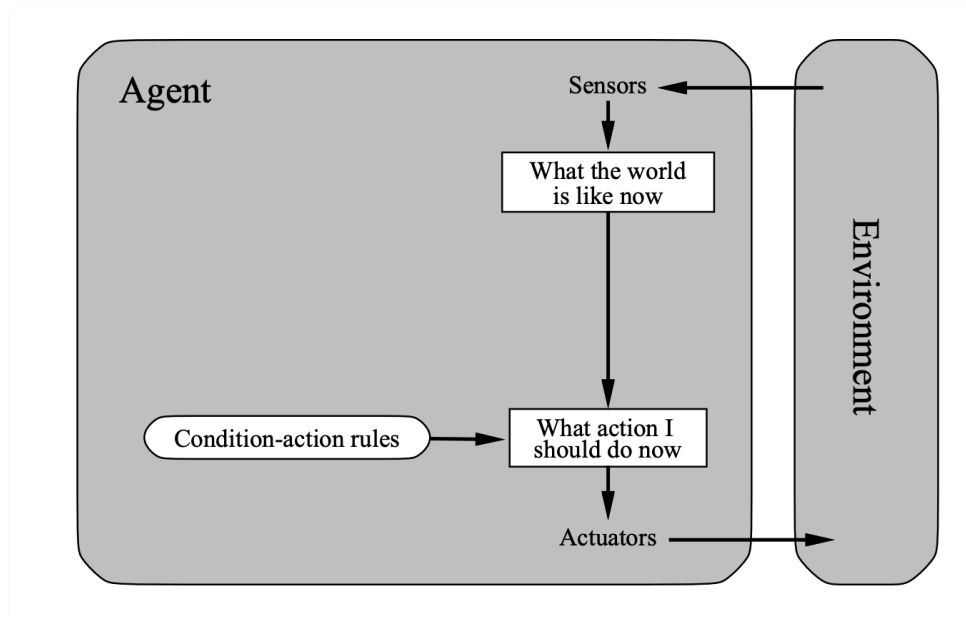


(Cumulative) Reward

$$R(\tau) = \sum_{t=0}^{H-1} r(s_t, a_t)$$

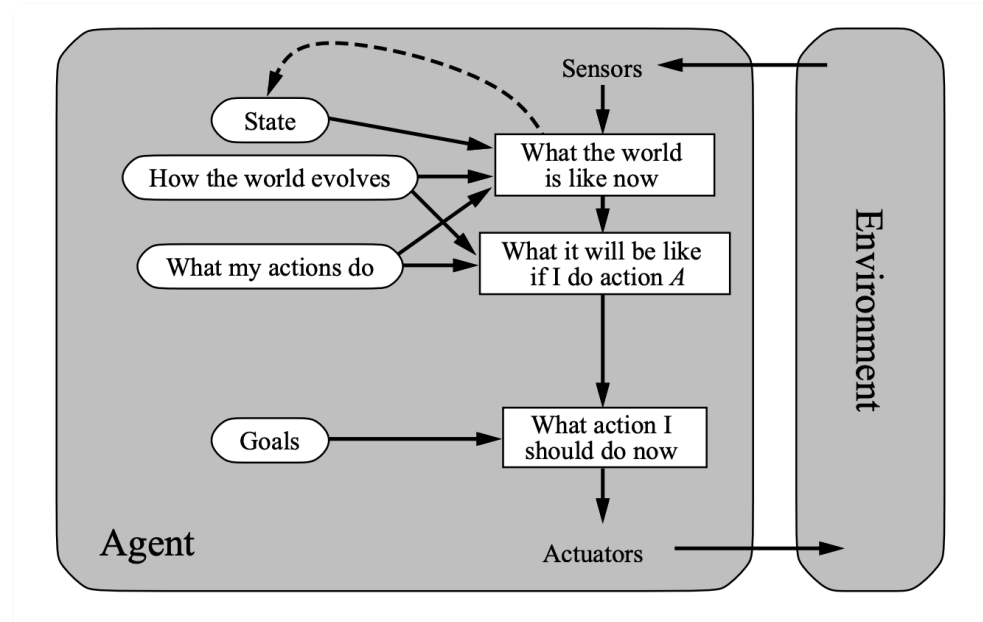
FROM PLAIN REACTIVE TO GOAL-CENTRIC AGENTS

Plain reactive



- The direct use of perceptions is often not possible because they require a lot of memory (e.g. video images).
- Inputs are therefore often interpreted before decisions are made.
- Condition - rules of action
- Forget the past

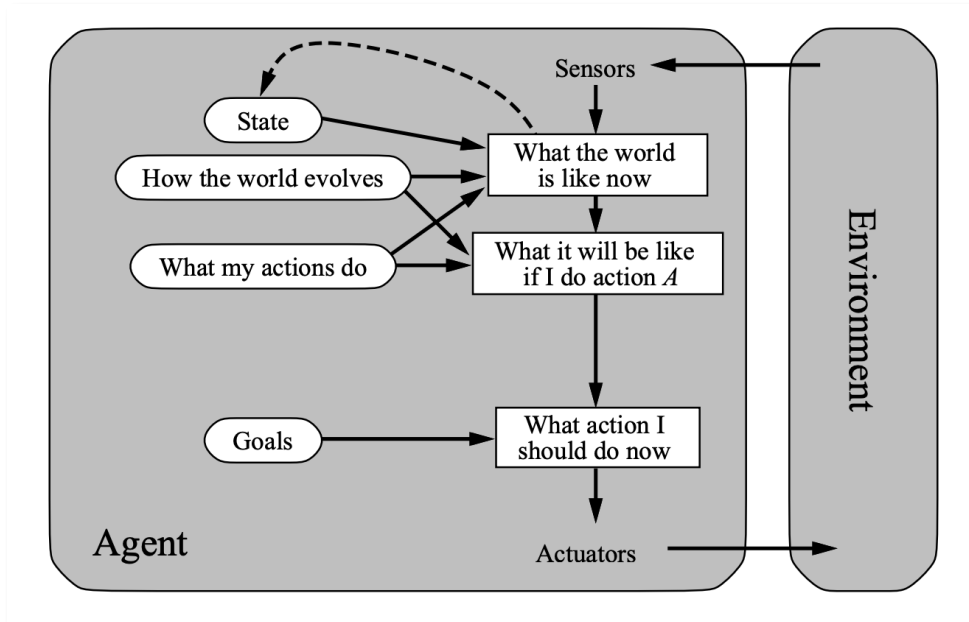
Goal-centric



- Often, perceptions alone are not enough to decide what to do.
- This is because the correct action depends on the given explicit goals (e.g., go in direction X).
- The model-based, goal-based agents use an explicit representation of goals and take them into account when choosing actions.

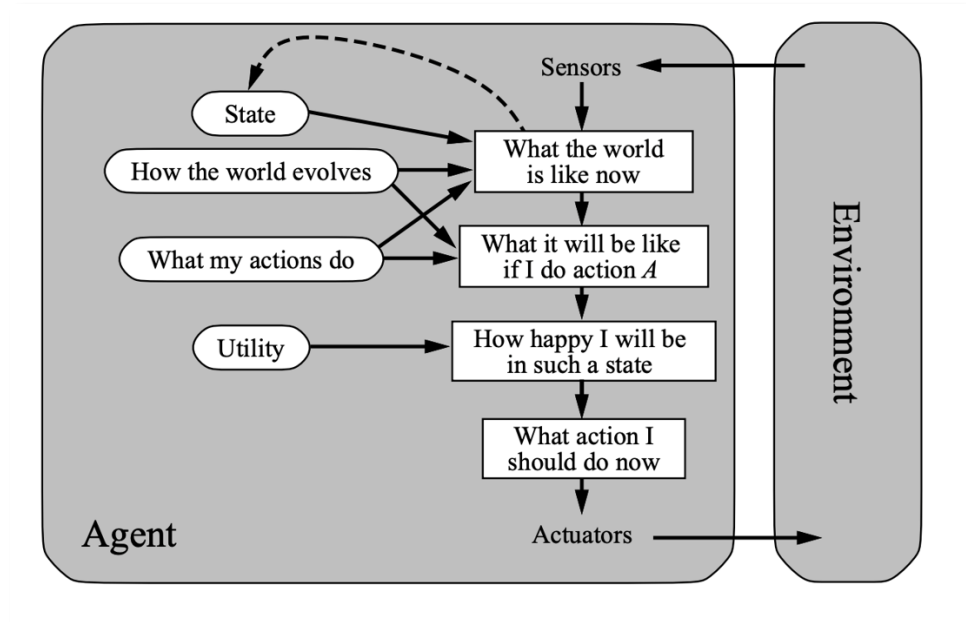
FROM GOAL-CENTRIC AGENTS TO UTILITY-CENTRIC AGENTS

Goal-centric



- Often, perceptions alone are not enough to decide what to do.
- This is because the correct action depends on the given explicit goals (e.g., go in direction X).
- The model-based, goal-based agents use an explicit representation of goals and take them into account when choosing actions.

Utility-centric



- Goal-oriented = binary decision (happy/not happy).
- Usually several possible actions.
- Then: Consider the expected benefit of an action to make a decision.
- Benefit = internal performance measure
- Utility function: maps a state (or sequence of states) to a real number.

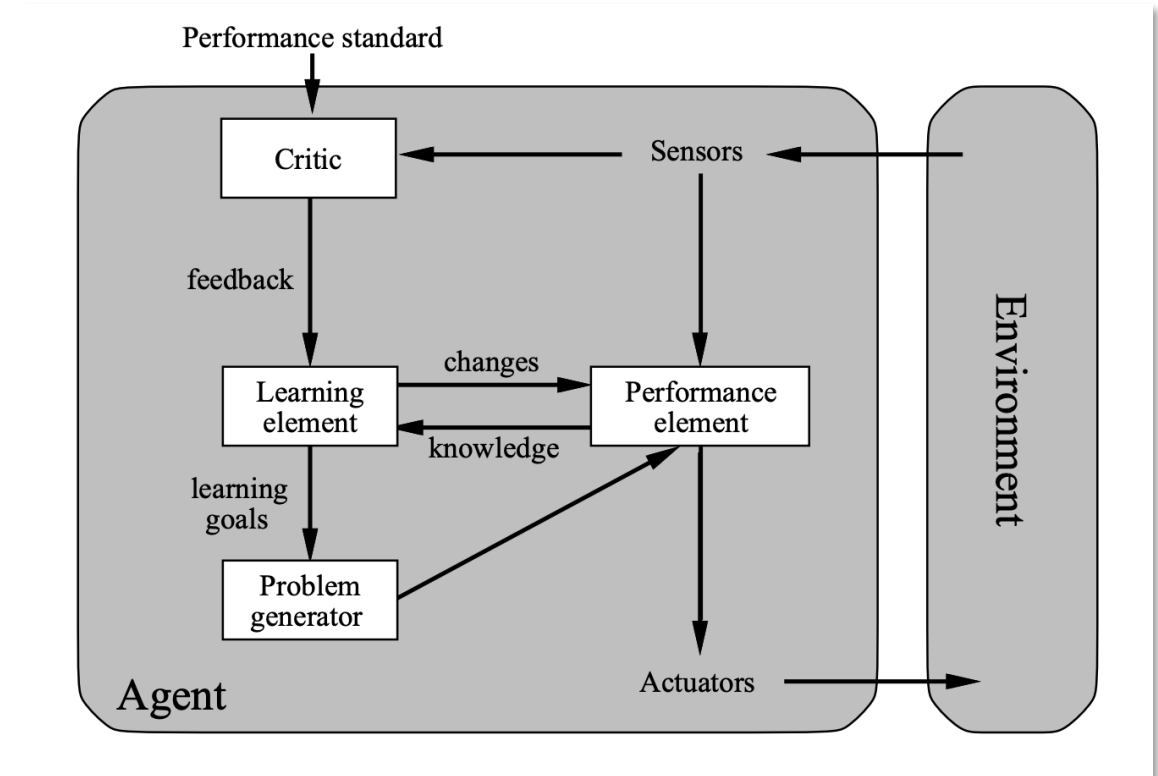
LEARNING AGENTS

Core properties

- Learning agents can become more competent over time.
- You can start with an initially empty knowledge base.
- You can work in initially unfamiliar environments.

Components

- Learning element (responsible for improvements)
- Performance element (must select external actions, our previous agent).
- Critic (determines how the performance element should be changed to perform better).
- Problem generator (suggests actions that lead to informative experiences).
- Punishment/reward direct feedback of agent behavior to learn benefits



ENVIRONMENTS

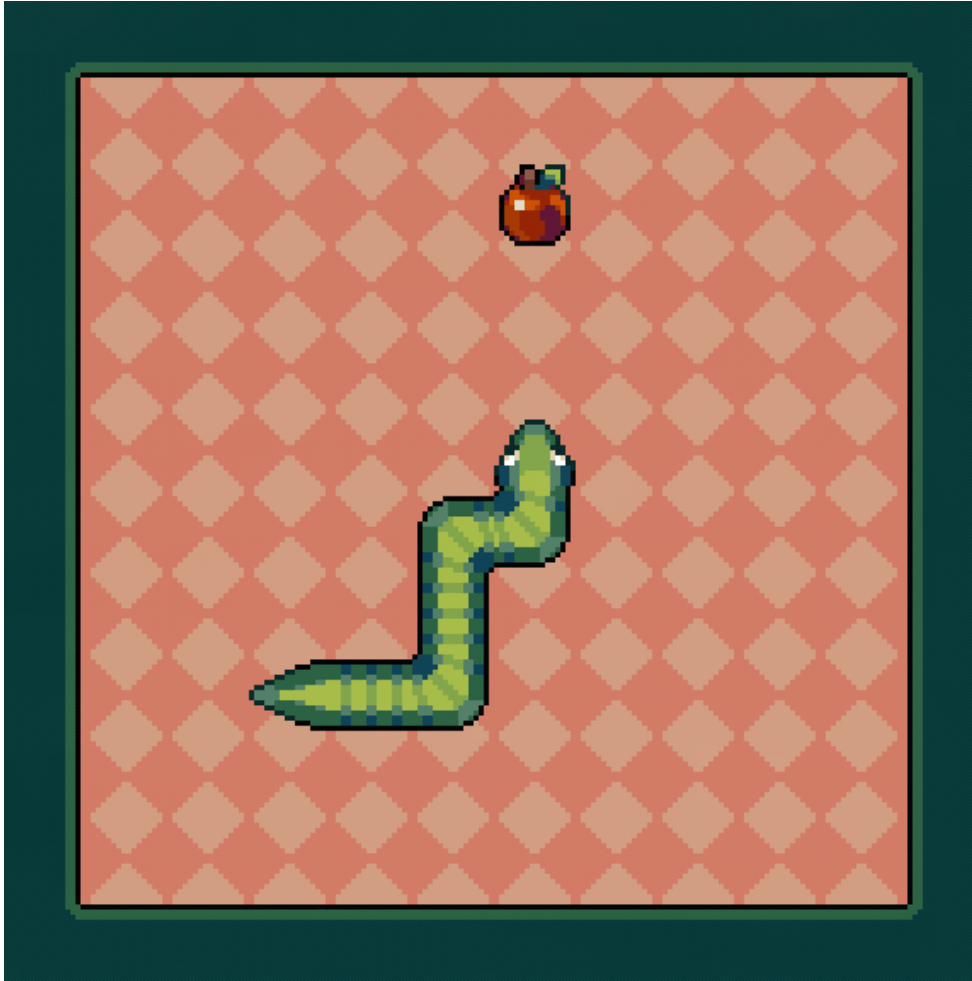
- Partially/completely observable: Is the entire state of the environment known at any point in time?
- Single / multiple agents: Opposing, cooperative, indifferent
- Deterministic / stochastic: If the state $Z(t+1) = f(Z(t), A)$ is always the same $\rightarrow$ deterministic
- Episodic / sequential: Episodic tasks are atomic and have no influence on later tasks (image classification)
- Static / dynamic: Does the environment change while the agent is planning?
- Discrete / continuous: states and inputs
- Known / unknown: Are all laws known, etc.



REINFORCEMENT LEARNING METHODS

- Value-based vs. policy-based
- Q-Learning
- Deep Q-Learning

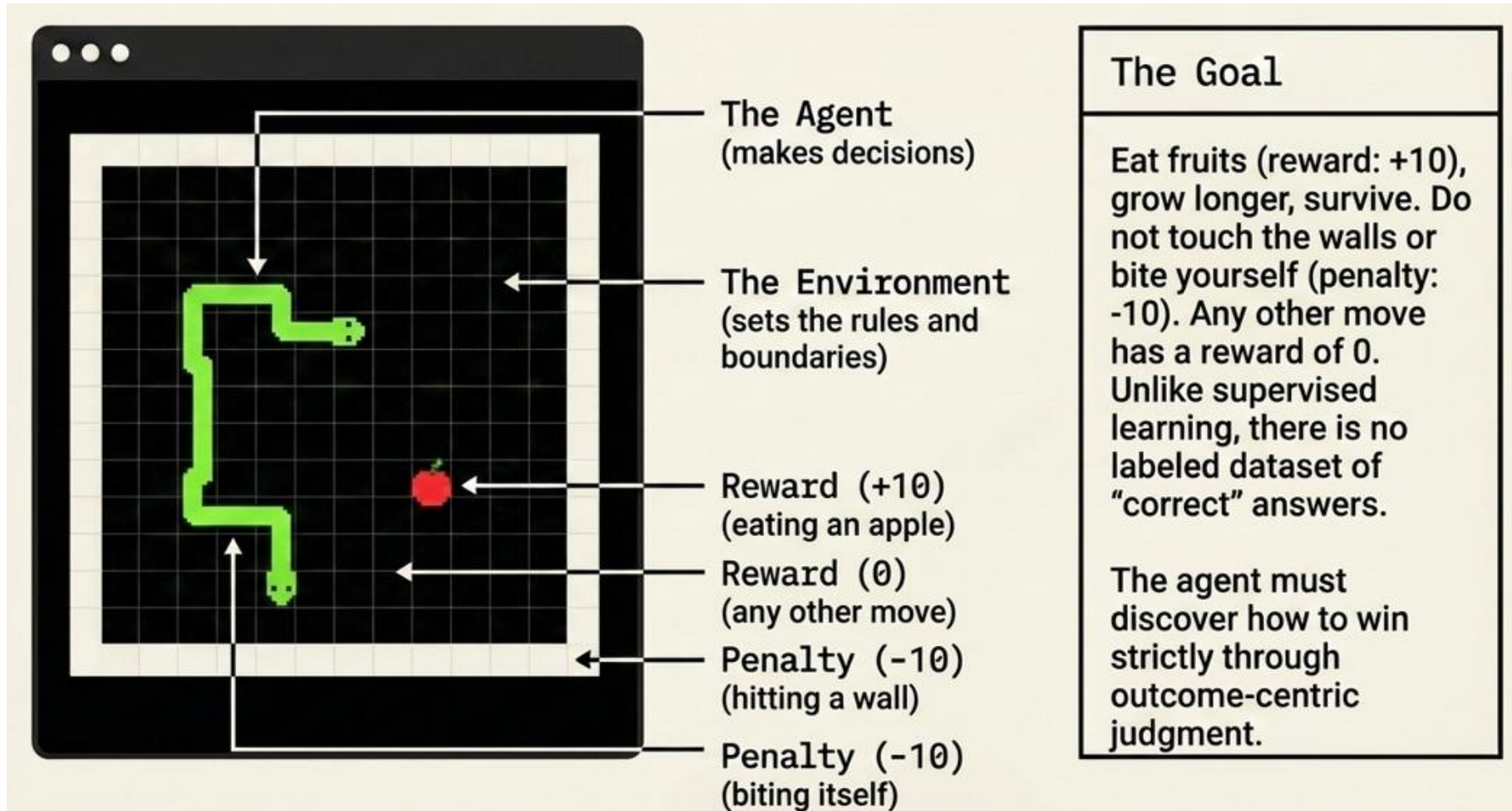
EXAMPLE: THE SNAKE GAME



How can we create a computer program that learns how to play that game without telling it about the correctness of any action it takes?

Goal: Eat as lot of **food** as possible without hitting any **wall** or **biting yourself**!

INTRODUCING THE COMPONENTS OF REINFORCEMENT LEARNING

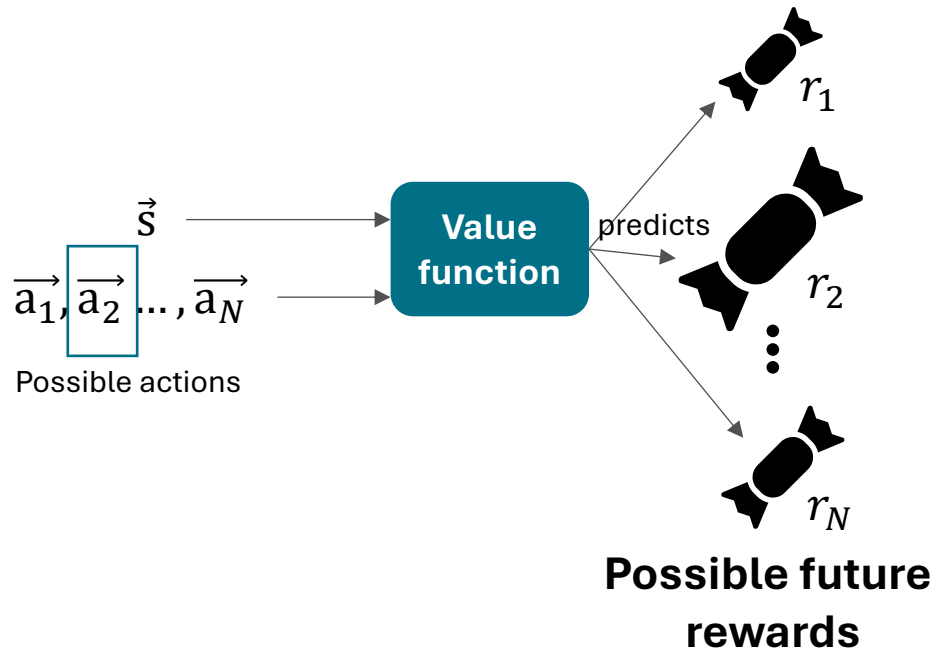


VALUE-BASED VS. POLICY-BASED METHODS

For motivating policy-based methods

Value-based

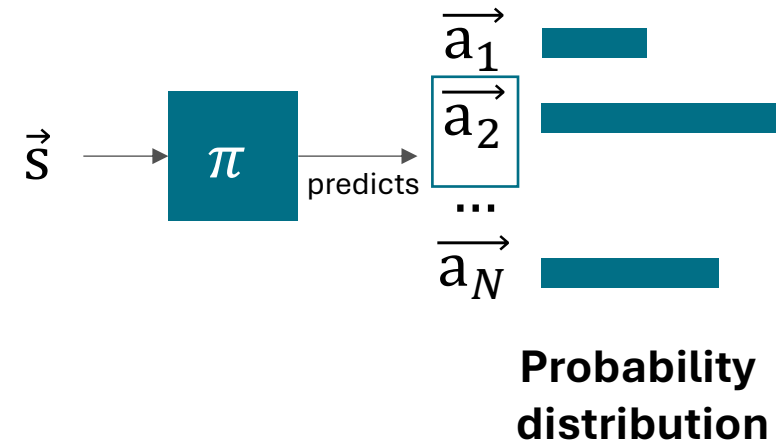
Learn a function to evaluate actions



Go for the biggest reward

Policy-based

Learn a function to select actions



Go for the most likely action

Q-LEARNING

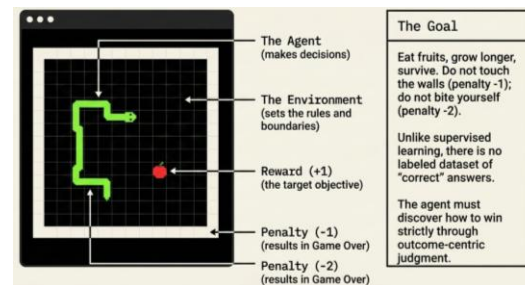
Q-Values

The maximum reward for given state and action

Principle

- Value-based approach based on a **Q-Table**
- Calculates the maximum expected **future reward** for each action at each state
- Choose the action with the **highest (expected) reward**

We stick to our example here ...



State

Actions

	Left	Right	Up	Down
Heading Left	0	0	0	1
Heading Right	0	0	1	0
Heading Up	-1	0	1	0
Heading Down	-1	0	0	0
Apple Left	2	0	0	0
Apple Right	0	5	0	0
Apple Up	1	0	0	0
Apple Down	0	0	0	0
...				

Note: States are typically complex and not atomic (e.g., a position in combination with information about nearby elements).

Q-LEARNING ALGORITHM

The Q-Learning algorithm is an iterative approach that updates Q-Values

1. Initialize Q-Table (with e.g., zeroes for all Q-Values).
2. Choose action a for state s (according to the best Q-value or randomly from all best Q-values)
3. Perform action a and receive new state s'
4. Measure reward R after action a
5. Update Q-Value with Bellman Equation ($Q_{\text{new}} = \dots$)

Q2: What is the Bellman equation??

State

Actions

	Left	Right	Up	Down
Heading Left	2	0	0	0
Heading Right	0	0	0	0
Heading Up	0	0	0	0
Heading Down	0	0	0	0
Apple Left	0	0	0	0
Apple Right	0	0	0	0
Apple Up	0	0	0	0
Apple Down	0	0	0	0
...				

Q1: After first update it might be unrealistic that any other action is taken in the same situation. What can we do?

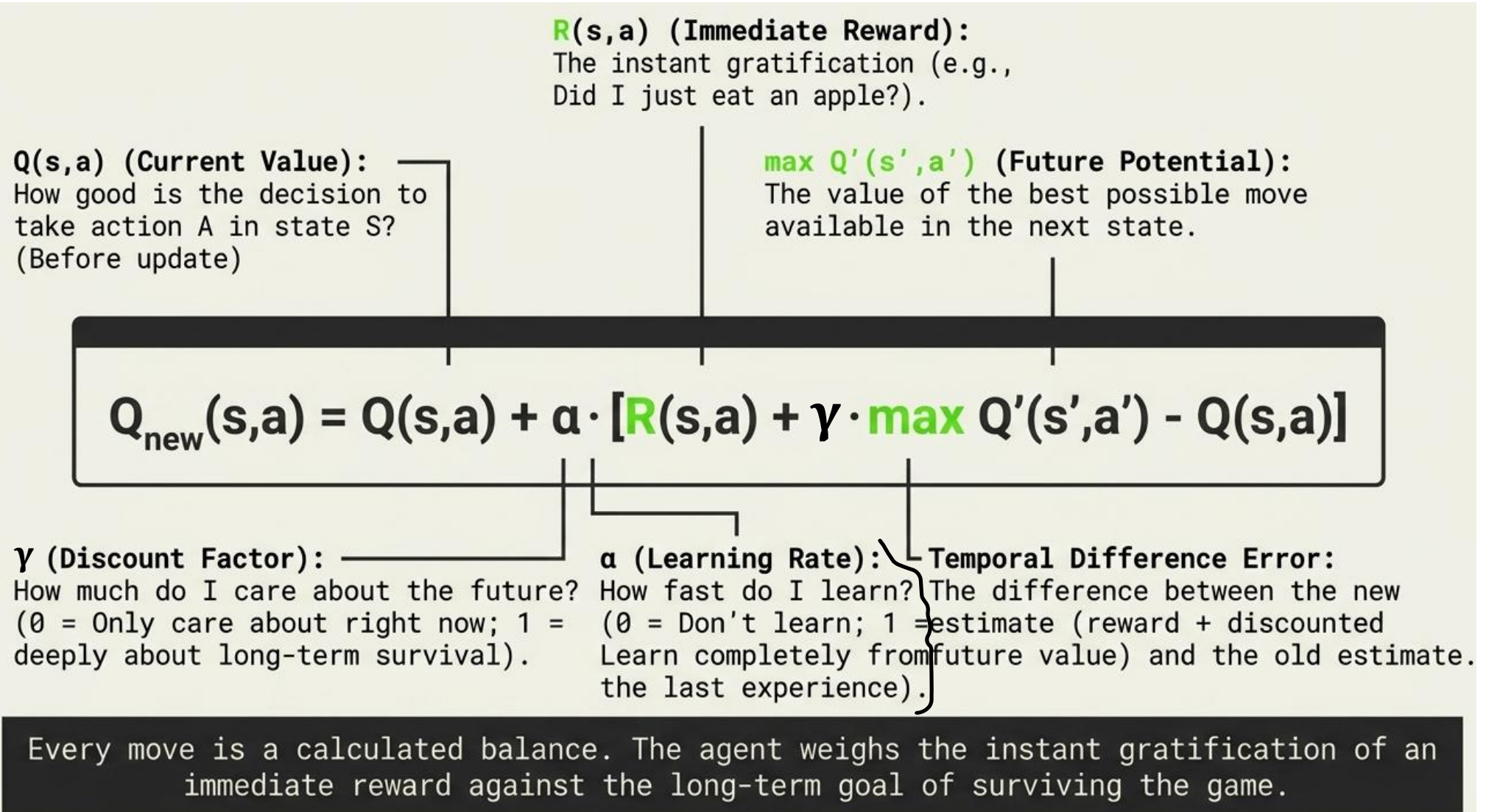
EXPLORATION VS. EXPLOITATION

Q1: After first update it might be unrealistic that any other action is taken in the same situation. What can we do?

Solution

- At the beginning choose action **randomly** so that the agent can **explore** the environment.
- The more training steps are done, the more we **reduce the random exploration** and **use exploitation instead**.
- This tradeoff is controlled by a parameter typically called ϵ (“epsilon”)

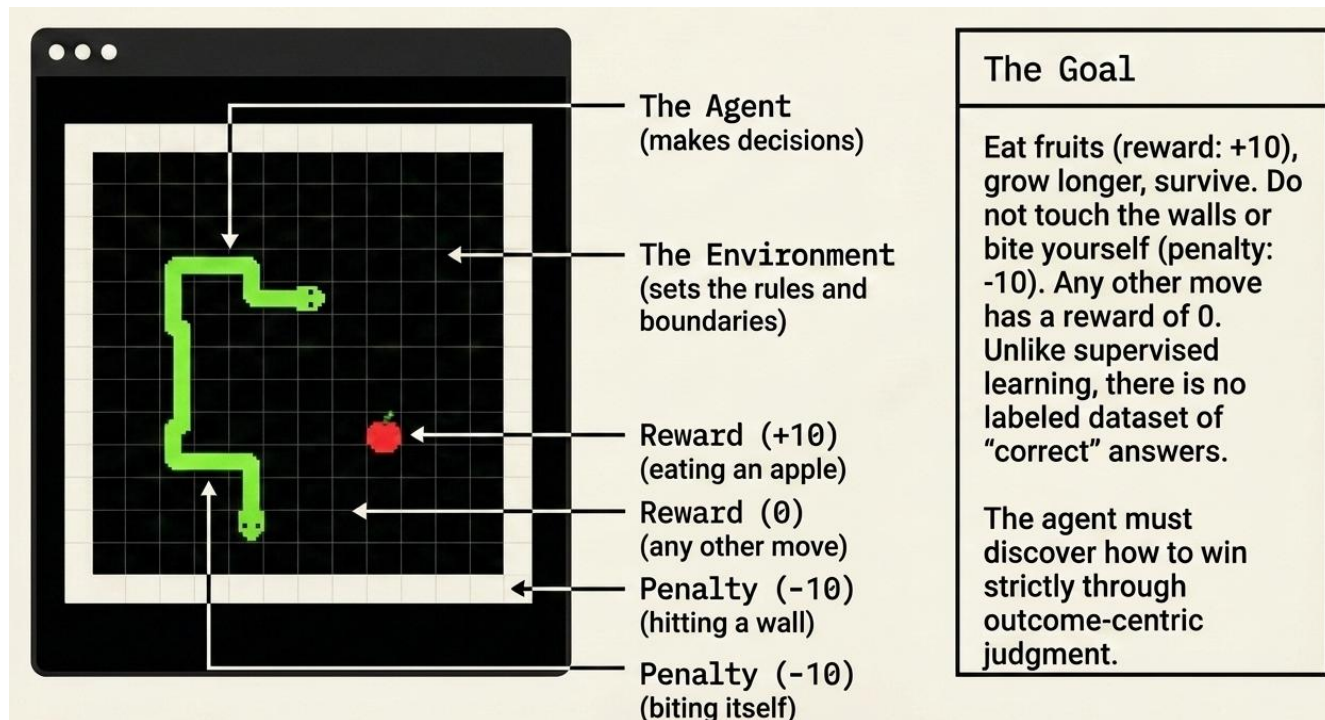
BELLMAN EQUATION



REWARDS

How do we define the rewards?

- Short answer: Based on handcrafted rules!
- Together with defining the states for the Q-Table this is the “trickiest” part in Q-Learning

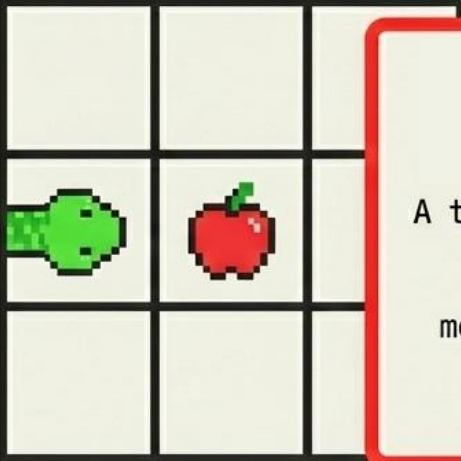


Snake eats apple: +10

Snake dies: -10

All other moves: 0

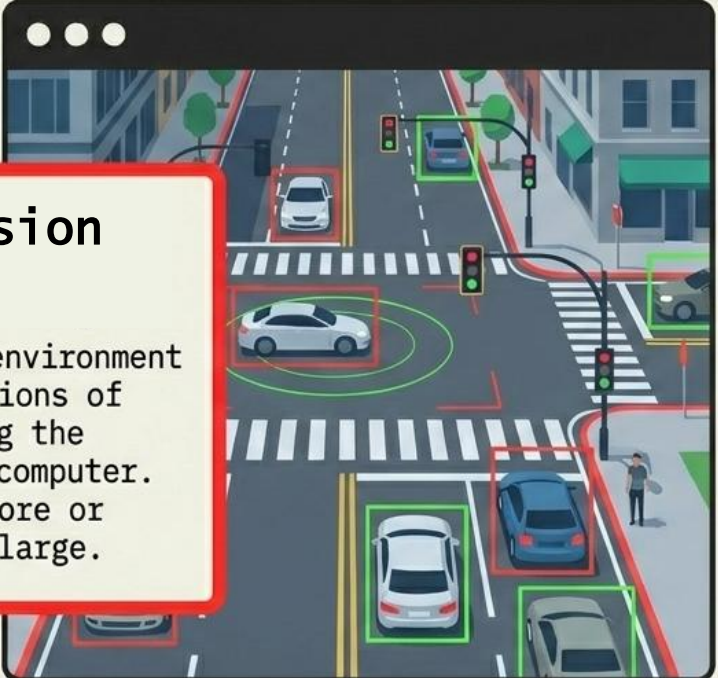
LIMITATIONS OF Q-LEARNING



Small, discrete states.
A few hundred rows.
Easily stored in memory.

State explosion issue

A table for a complex environment would require trillions of rows—exceeding the memory limits of any computer. The agent cannot store or learn a table that large.



Millions of continuous pixels.
Infinite variations.

How can we solve that state-explosion issue?

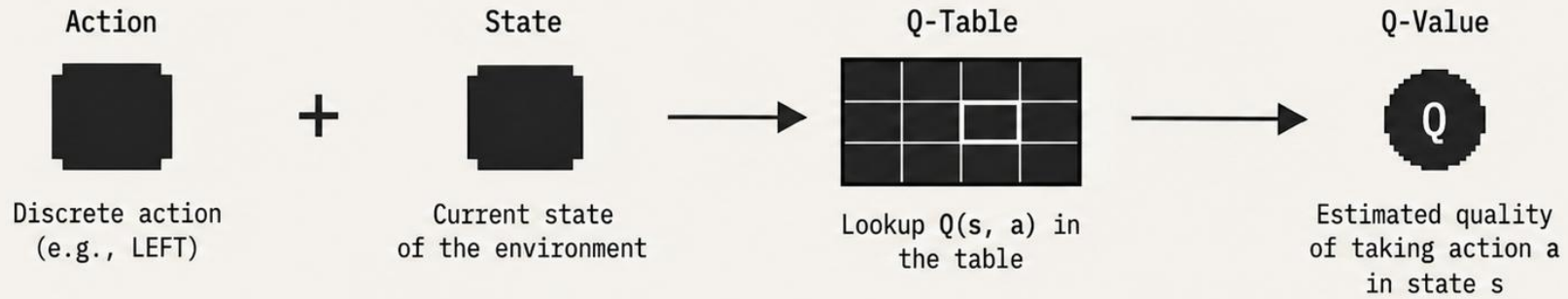


DEEP-Q-LEARNING

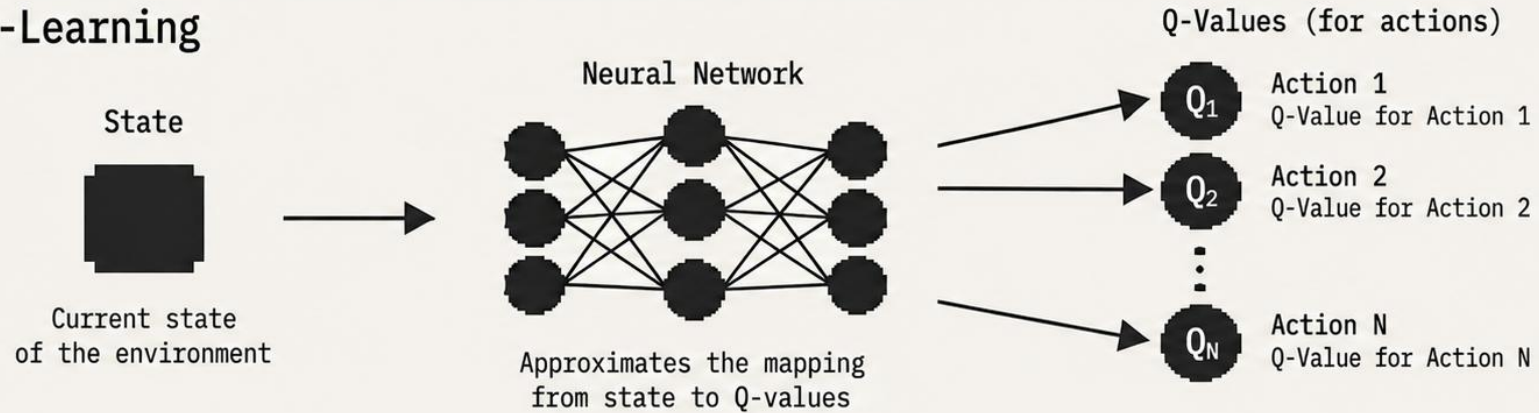
- Changes to Q-Learning
- A small step to policy-centric approaches

A VERY SIMPLE CHANGE ...

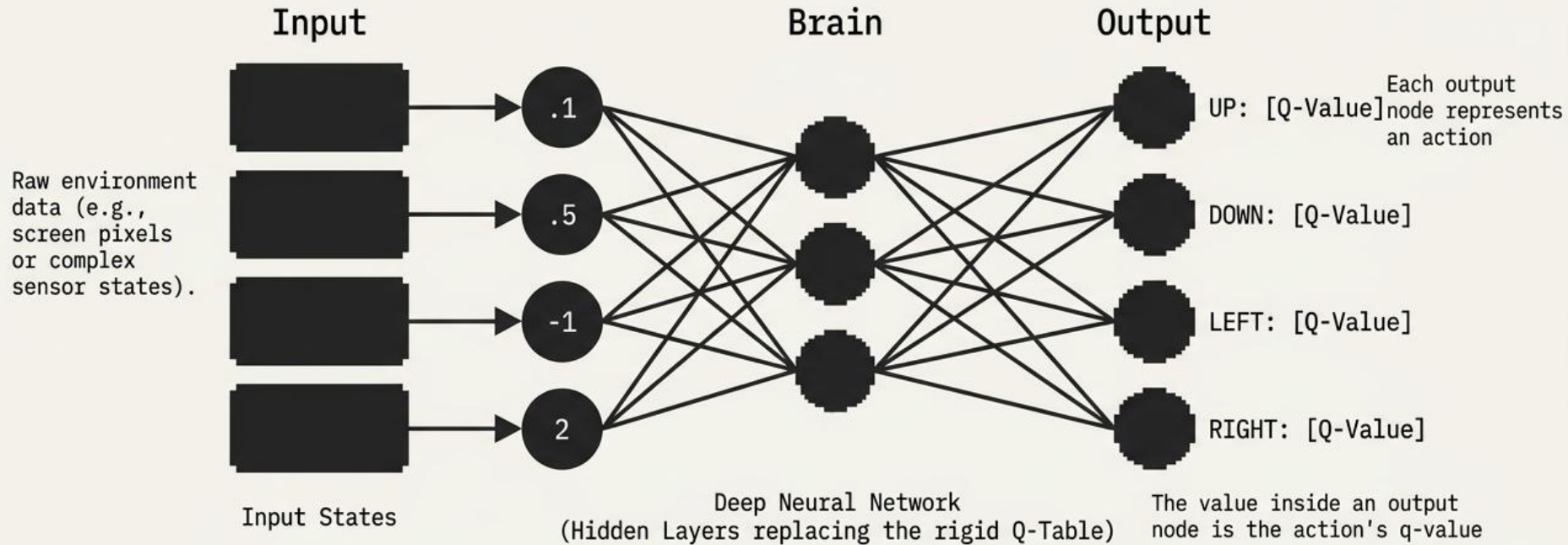
Q-Learning



Deep Q-Learning



HOW DOES THAT NEURAL NETWORK OPERATE?



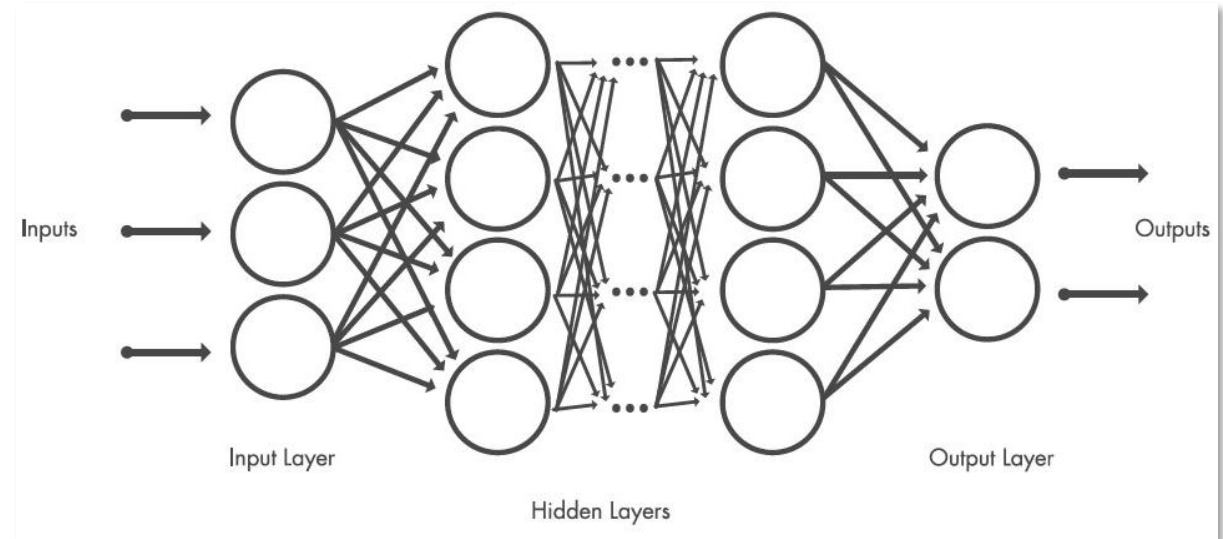
**Why is DQN
still a value-
based
approach?**

We throw away the table. By replacing the rigid 'cheat sheet' with a flexible Neural Network, the agent no longer memorizes exact situations. Instead, it generalizes patterns. It learns that 'approaching a solid object' is bad, regardless of exactly where it appears on the screen.

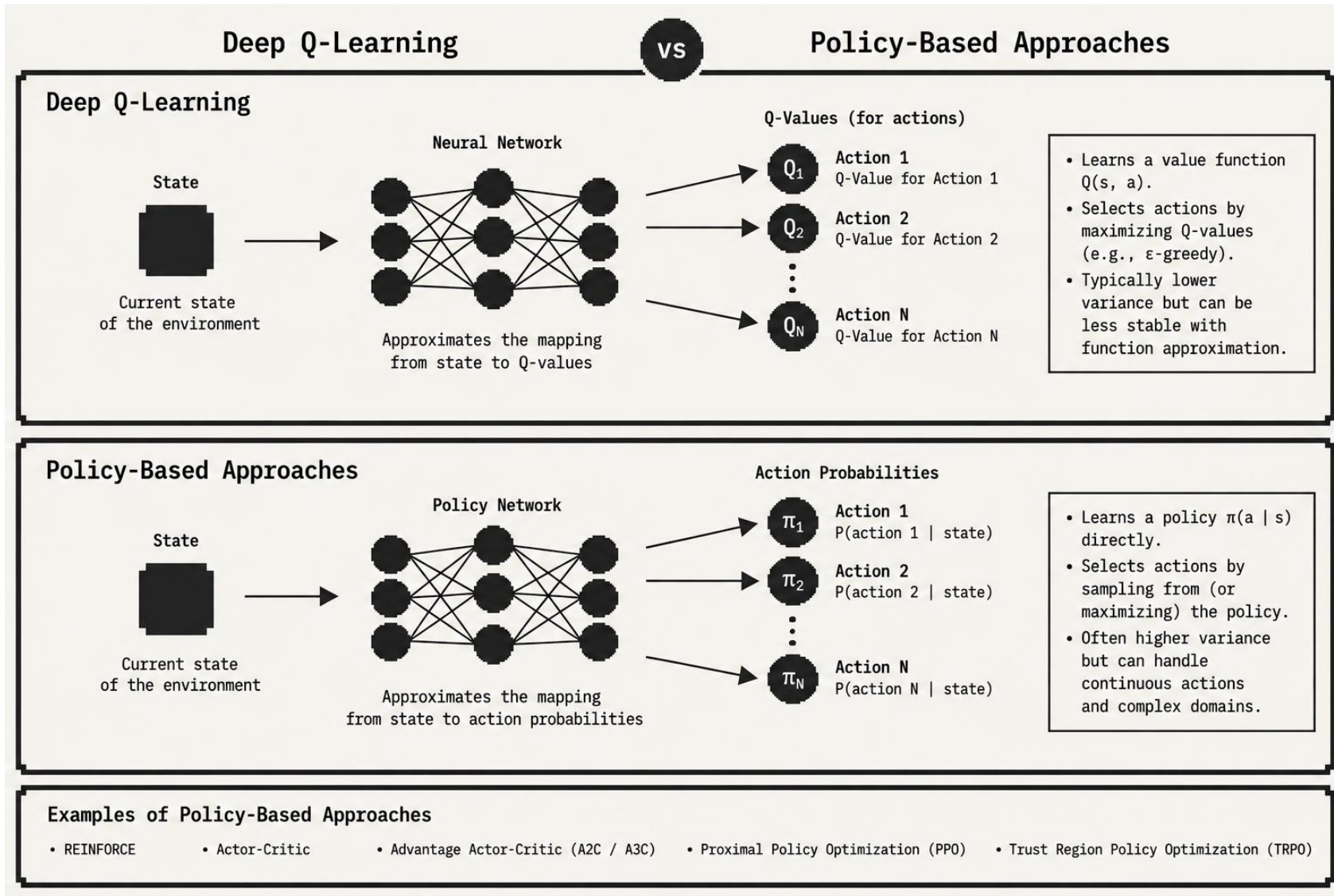
DEEP Q-LEARNING ALGORITHM

The Deep Q-Learning algorithm is an iterative approach that updates **Q-Values** the weights of a neural network

- 1. Initialize Q-Table (with e.g., zeroes for all Q-Values): a neural network
- 2. Choose action a for state s (according to the best Q-value or randomly from all best Q-values)
- 3. Perform action a and receive new state s'
- 4. Measure reward R after action a
- 5. Update weights of the Neural Network



A SMALL STEP TO POLICY-CENTRIC METHODS





HANDS-ON

- Gymnasium
- Implementing LunarLander

IMPLEMENTING REINFORCEMENT LEARNING WITH GYMNASIUM

- The **environment** is created with Gymnasium using `gym.make("LunarLander-v3", render_mode="rgb_array")`, which provides an 8-dimensional continuous state (position, velocity, angles) and discrete action space for thrust control.
- The agent interacts with the environment in a loop: at each step it observes the current state (`obs`), selects an action using the trained DQN policy, applies it via `env.step(action)`, and receives the next state, reward, and termination signals (`terminated`, `truncated`).
- The Deep Q-Network (DQN) is implemented via Stable-Baselines3 as `DQN("MlpPolicy", env, ...)`, where a multi-layer perceptron approximates the Q-function $Q(s, a)$, mapping states to action-value estimates; the action with the highest Q-value is selected during inference (`model.predict`).
- During training, DQN uses experience replay and a target network: transitions (s, a, r, s') are stored in a replay buffer, and the network is updated using mini-batches to minimize the difference between predicted Q-values and target values $r + \gamma \max_{a'} Q(s', a')$.
- The rollout loop records frames via `env.render()` before each step and stores them in a list, enabling post-hoc video generation while also tracking cumulative episode reward for performance evaluation.



Complete notebook available (soon)
in our Moodle course!

QUESTIONS?





DONE.

95028 Hof
Alfons-Goppel-Platz 1
Phone +49 9281 409-3000
Fax +49 9281 409-4000
mail@hof-university.de
www.hof-university.de